



Cryptanalyse

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction à la cryptanalyse : histoire et classification des attaques

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définition

La **cryptanalyse** est la discipline scientifique qui étudie les méthodes permettant de retrouver, en tout ou partie, l'information protégée par un système cryptographique — le message clair, la clé secrète, ou toute information partielle qui affaiblit la sécurité annoncée — sans disposer légitimement de la clé de déchiffrement. Elle constitue le pendant offensif de la cryptographie : là où la cryptographie conçoit des mécanismes de protection, la cryptanalyse les met à l'épreuve pour en révéler les faiblesses éventuelles.

Ces deux disciplines ne s'opposent pas, elles progressent **ensemble et se nourrissent mutuellement**. Un algorithme cryptographique n'est jamais considéré comme fiable simplement parce que son concepteur l'affirme : il ne gagne la confiance de la communauté scientifique qu'après avoir résisté, pendant plusieurs années et face à un examen public et intensif, aux meilleures techniques de cryptanalyse connues. C'est le sens profond du **principe de Kerckhoffs** (vu au module *Cryptographie*) : la sécurité d'un système ne doit reposer que sur le secret de la clé, jamais sur le secret de l'algorithme. Un algorithme publié, largement étudié, et toujours non cassé après des années d'efforts (AES, SHA-256) inspire objectivement plus confiance qu'un algorithme secret ou « maison », dont personne n'a pu vérifier la robustesse — car un algorithme fondé sur le secret de sa conception s'effondre entièrement le jour où ce secret est découvert ou rétro-ingénieré.

Il est important de noter que la cryptanalyse, telle qu'enseignée dans ce chapitre, vise un objectif strictement **académique et défensif** : comprendre les principes qui ont permis de casser des systèmes historiques ou théoriques permet de mieux concevoir, choisir et déployer des systèmes modernes robustes, et de reconnaître les signaux d'alerte d'une implémentation dangereuse.

2. Repères historiques

L'histoire de la cryptanalyse est aussi ancienne que celle de la cryptographie elle-même ; chaque avancée cryptographique majeure a fini, tôt ou tard, par susciter une réponse cryptanalytique qui a fait progresser les deux domaines.

- **Antiquité à Renaissance** : le chiffrement de César (décalage fixe de l'alphabet) et, plus généralement, les chiffres de substitution mono-alphabétique dominent les communications secrètes pendant des siècles. Ils sont cassés dès le IXe siècle par le savant arabe **Al-Kindi**, dans son traité sur le déchiffrement des messages cryptographiques, où il formalise pour la première fois l'**analyse fréquentielle** : chaque langue possède une distribution statistique caractéristique de ses lettres, et cette distribution « survit » à une simple substitution.
- **XVIe siècle** : le chiffre de **Vigenère**, qui applique un décalage différent à chaque position du texte selon une clé répétée, est longtemps surnommé « le chiffre indéchiffrable » (*le chiffre indéchiffrable*) car il résiste, en apparence, à l'analyse fréquentielle directe. Il faudra attendre le XIXe siècle pour que **Friedrich Kasiski** (1863) puis, indépendamment et plus tôt mais sans publication, **Charles Babbage**, développent des méthodes systématiques pour le casser, en s'appuyant sur la répétition périodique de la clé.
- **Seconde Guerre mondiale** : la cryptanalyse de la machine électromécanique **Enigma**, utilisée par l'Allemagne nazie, est menée d'abord par les mathématiciens polonais (Marian Rejewski et son équipe, dès les années 1930), puis reprise et considérablement étendue par les équipes britanniques de **Bletchley Park**, sous l'impulsion notamment d'**Alan Turing**. Cet épisode constitue un tournant historique double : il marque la naissance de la cryptanalyse industrialisée, mécanisée à grande échelle, et il est directement à l'origine de certains fondements théoriques et pratiques de l'informatique moderne (les « bombes » électromécaniques de Turing préfigurant le calcul automatisé).
- **Ère moderne (seconde moitié du XXe siècle à aujourd'hui)** : l'apparition des chiffrements par blocs numériques (DES en 1977) suscite le développement d'attaques structurelles sophistiquées comme la **cryptanalyse différentielle** et la **cryptanalyse linéaire** (détaillées au chapitre 3), tandis que l'essor de la cryptographie asymétrique (RSA, 1977) ouvre un nouveau front d'attaques mathématiques fondées sur la factorisation d'entiers et le logarithme discret (chapitre 5). Plus récemment, les **attaques par canaux auxiliaires** (chapitre 6) ont montré que la sécurité d'un système ne peut se réduire à la robustesse théorique de son algorithme : l'implémentation physique compte tout autant.

3. Classification des attaques selon les informations disponibles

Pour évaluer objectivement la robustesse d'un système cryptographique, il est essentiel de préciser **quel modèle d'attaquant** on considère : quelles informations cet attaquant peut-il obtenir, et avec quelles capacités d'interaction avec le système ? Cette classification, standard en cryptanalyse, ordonne les modèles du plus faible (moins d'information pour l'attaquant) au plus fort.

Type d'attaque	Information disponible à l'attaquant	Exemple illustratif
Ciphertext-only (texte chiffré seul)	uniquement des messages chiffrés interceptés, sans aucun texte clair correspondant	interception passive de communications chiffrées sur un réseau
Known-plaintext (texte clair connu)	un ou plusieurs couples (texte clair, texte chiffré correspondant), obtenus par exemple parce qu'une partie du contenu est prévisible ou a fuité par ailleurs	un en-tête de fichier standardisé, toujours identique, dont la version chiffrée est observée
Chosen-plaintext (texte clair choisi)	l'attaquant peut soumettre des textes clairs de son choix au système et observer les chiffrés correspondants	un attaquant qui interagit avec un service en ligne acceptant de chiffrer des données qu'il fournit
Chosen-ciphertext (texte chiffré choisi)	l'attaquant peut, en plus, soumettre des textes chiffrés de son choix et observer le résultat de leur déchiffrement (à l'exception du message cible)	un système qui répond différemment selon que le déchiffrement d'un texte chiffré modifié réussit ou échoue

Ce classement traduit une progression de la puissance du modèle d'attaquant : de *ciphertext-only* à *chosen-ciphertext*, l'attaquant dispose de capacités d'interaction de plus en plus fortes avec le système cible. Un algorithme jugé robuste doit résister au modèle le plus exigeant qui soit **réaliste** dans son contexte d'usage : par exemple, un schéma de chiffrement utilisé dans un protocole interactif (comme TLS) doit impérativement résister aux attaques à texte chiffré choisi, car un attaquant actif sur le réseau peut effectivement injecter des messages modifiés et observer les réactions du système (c'est précisément ce mécanisme qu'exploitent les attaques par *padding oracle*, vues au chapitre 6). Un algorithme qui ne résiste qu'au modèle *ciphertext-only* le plus faible est donc, par construction, insuffisant pour la plupart des usages pratiques modernes.

4. Classification des attaques selon la méthode

Une seconde façon, complémentaire, de classer les attaques cryptanalytiques consiste à s'intéresser non plus à ce que l'attaquant observe, mais à **la nature de la faiblesse exploitée**.

- **Attaques statistiques** : elles exploitent des propriétés statistiques résiduelles du texte clair (fréquence des lettres, des mots, redondance du langage naturel) qui n'ont pas été suffisamment « effacées » par le chiffrement. L'analyse fréquentielle contre les substitutions classiques (chapitre 2) en est l'exemple archétypal.
- **Attaques par force brute (recherche exhaustive)** : elles consistent à tester systématiquement toutes les clés possibles jusqu'à trouver la bonne, sans exploiter aucune faiblesse structurelle de l'algorithme. Leur coût croît exponentiellement avec la taille de la clé, ce qui rend cette approche impraticable dès que l'espace de clés est suffisamment grand — mais redoutablement efficace lorsque cet espace est trop restreint (voir le chiffre de César, chapitre 2, cassable en 25 essais).
- **Attaques structurelles/mathématiques** : elles exploitent une faiblesse mathématique propre à la conception de l'algorithme — une hypothèse calculatoire qui s'avère plus faible que prévu (factorisation d'entiers, logarithme discret), ou un biais statistique interne à la structure de l'algorithme (biais différentiels ou linéaires dans un chiffrement par blocs, chapitre 3).
- **Attaques par canaux auxiliaires (side-channel)** : elles n'attaquent pas l'algorithme mathématique lui-même, mais exploitent des informations physiques involontairement révélées par son **implémentation concrète** — temps d'exécution, consommation électrique, rayonnement électromagnétique (chapitre 6). Elles rappellent qu'une preuve mathématique de sécurité ne protège pas, à elle seule, contre une implémentation défailante.
- **Attaques par erreur d'implémentation ou de configuration** : elles exploitent des erreurs pratiques dans la façon dont un système cryptographique est déployé — générateur de nombres aléatoires de mauvaise qualité, réutilisation d'une clé ou d'un nonce, choix d'un mode opératoire inapproprié. En pratique, c'est de très loin la **source la plus fréquente** de compromission de systèmes cryptographiques réels : l'algorithme mathématique sous-jacent est presque toujours plus robuste que son utilisation concrète.

Cette seconde classification n'est pas exclusive de la première : une attaque donnée se décrit généralement à la fois par le modèle d'information qu'elle suppose (section 3) et par la nature de la faiblesse qu'elle exploite (section 4). Les chapitres suivants de ce module reprennent systématiquement cette double grille de lecture pour chaque famille de systèmes cryptographiques étudiée.

5. Notion de sécurité et de coût d'une attaque

Un principe fondamental, souvent contre-intuitif pour qui découvre la cryptographie, est qu'un système cryptographique n'est presque jamais qualifié de « sûr » parce qu'il serait mathématiquement incassable dans l'absolu. Il est qualifié de sûr parce que **le coût de la meilleure attaque connue** — exprimé en temps de calcul, en mémoire nécessaire, ou en nombre de messages requis — dépasse très largement ce qu'un attaquant réaliste peut mobiliser, aujourd'hui et dans un avenir prévisible.

Cette notion de coût se mesure typiquement en ordres de grandeur d'opérations élémentaires : on parle par exemple d'une sécurité « de 128 bits » pour signifier que la meilleure attaque connue nécessite de l'ordre de 2^{128} opérations, un nombre si colossal qu'il dépasse très largement les capacités de calcul cumulées de l'humanité entière sur des échelles de temps raisonnables, même en tenant compte des progrès attendus du matériel informatique classique.

La conséquence directe de ce principe est que la sécurité cryptographique est **relative et évolutive**, et non absolue et figée :

- Un algorithme jugé sûr aujourd'hui peut devenir vulnérable demain, soit parce qu'une **nouvelle technique cryptanalytique** est découverte et réduit le coût de la meilleure attaque connue (c'est arrivé à DES avec la cryptanalyse différentielle, puis avec la simple augmentation de la puissance de calcul disponible), soit parce que la **puissance de calcul disponible** augmente au point de rendre praticable une attaque autrefois hors de portée (c'est ce qui a rendu la force brute contre DES 56 bits réalisable en quelques heures avec du matériel spécialisé moderne, alors qu'elle était impraticable en 1977).
- Une menace future bien identifiée peut également motiver une transition anticipée : c'est le cas de la menace, encore hypothétique mais prise très au sérieux, que représenterait un ordinateur quantique suffisamment puissant exécutant l'algorithme de Shor contre RSA et les schémas fondés sur le logarithme discret (approfondi au chapitre 5), qui justifie dès aujourd'hui la standardisation d'algorithmes post-quantiques.

Cette relativité de la sécurité justifie la pratique, essentielle en cryptographie appliquée, de prévoir une **marge de sécurité** confortable (taille de clé, nombre de tours d'un algorithme par blocs) au-delà du strict minimum théoriquement nécessaire au moment de la conception, et de suivre dans la durée l'évolution des connaissances cryptanalytiques et des capacités de calcul, afin d'anticiper les migrations nécessaires avant qu'une faiblesse ne devienne exploitable en pratique.

6. Méthodologie générale d'une cryptanalyse

Au-delà des techniques spécifiques présentées dans les chapitres suivants, une démarche méthodologique commune structure toute analyse cryptanalytique rigoureuse, qu'elle porte sur un chiffrement historique simple ou sur un algorithme moderne complexe :

1. **Comprendre précisément l'algorithme et son mode d'utilisation.** Il ne suffit pas de connaître l'algorithme de base : le mode opératoire (chapitre 3), la gestion des clés, le protocole englobant font partie intégrante du système à analyser, et sont souvent la source réelle des faiblesses exploitables.
2. **Identifier le modèle d'attaque réaliste.** Quelles informations un attaquant peut-il effectivement obtenir dans le contexte d'usage considéré (section 3) ? Un modèle trop optimiste conduirait à sous-estimer le risque ; un modèle irréaliste conduirait à des conclusions sans portée pratique.
3. **Rechercher des biais exploitables**, qu'ils soient statistiques (redondance du texte clair), structurels (propriété mathématique de l'algorithme) ou liés à l'implémentation (erreur de configuration, fuite d'information physique).
4. **Évaluer le coût de l'attaque** — complexité en temps, en mémoire, quantité de données nécessaires — et le comparer à un budget attaquant réaliste dans le contexte considéré (section 5). Une attaque « théorique » de complexité 2^{100} n'a, à ce jour, aucune portée pratique, même si elle est scientifiquement intéressante en ce qu'elle réduit la marge de sécurité par rapport à la borne de force brute théorique de 2^{128} .
5. **Le cas échéant, valider par une preuve de concept contrôlée**, sur un exemple réduit ou simulé, strictement dans un cadre académique et légal — jamais contre un système réel sans autorisation explicite.

À retenir

- La cryptanalyse et la cryptographie évoluent en miroir : un algorithme n'est validé que par une résistance prouvée à la cryptanalyse publique, jamais par la seule affirmation de son concepteur (principe de Kerckhoffs).
- Le modèle d'attaque (ciphertext-only à chosen-ciphertext) détermine la puissance réaliste de l'attaquant ; un système doit résister au modèle le plus fort qui soit pertinent dans son contexte d'usage.
- La sécurité cryptographique se mesure en coût d'attaque (temps, mémoire, données), pas en impossibilité mathématique absolue ; elle est donc relative et évolue avec les connaissances et la puissance de calcul disponibles.
- En pratique, la majorité des compromissions viennent d'erreurs d'implémentation ou de configuration, pas de failles mathématiques dans l'algorithme lui-même.

Chapitre 2 — Cryptanalyse des chiffrements classiques

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Rappel : chiffrement de César et substitution mono-alphabétique

Le **chiffre de César** est l'un des plus anciens systèmes de chiffrement connus : chaque lettre du texte clair est décalée d'un nombre fixe de positions dans l'alphabet, ce nombre constituant la clé (une valeur entière comprise entre 0 et 25 pour l'alphabet latin). Mathématiquement, si l'on numérote les lettres de A à Z de 0 à 25, le chiffrement d'une lettre m avec une clé k s'écrit $c = (m + k) \bmod 26$, et le déchiffrement $m = (c - k) \bmod 26$.

La **substitution mono-alphabétique** généralise ce principe : au lieu d'un simple décalage, chaque lettre de l'alphabet clair est remplacée par une autre lettre selon une **permutation quelconque** de l'alphabet, fixée une fois pour toutes pour l'ensemble du message. L'espace de clés est alors considérablement plus grand que celui du chiffre de César : il existe $26!$ (factorielle de 26), soit environ 4×10^{26} permutations possibles de l'alphabet, ce qui rend une attaque par force brute totalement impraticable en apparence. Pourtant, comme le montre la section suivante, la taille de l'espace de clés ne suffit pas à garantir la sécurité d'un chiffrement si sa structure laisse subsister une faiblesse exploitable par une autre méthode.

2. Analyse fréquentielle

Chaque langue naturelle possède une **distribution statistique caractéristique** de la fréquence d'apparition de ses lettres, qui reste remarquablement stable d'un texte suffisamment long à un autre. En français, par exemple, la lettre **E** apparaît avec une fréquence d'environ 14,7 %, suivie par des lettres comme **A** ($\approx 7,6$ %), **S** ($\approx 7,9$ %) ou **I** ($\approx 7,5$ %), tandis que des lettres comme **W**, **K** ou **Z** sont beaucoup plus rares. Cette régularité statistique, propre à chaque langue, est le point de départ de l'analyse fréquentielle, formalisée dès le IX^e siècle par le savant arabe Al-Kindi (chapitre 1).

Le principe clé est le suivant : une substitution mono-alphabétique, en remplaçant systématiquement chaque lettre par une autre lettre fixe, **ne modifie pas la distribution statistique**, elle se contente de la « déplacer » d'une lettre vers une autre. Autrement dit, si E est la lettre la plus fréquente du texte clair, la lettre qui la remplace dans le texte chiffré sera très probablement la lettre la plus fréquente du texte chiffré. Cette invariance structurelle transforme un espace de clés gigantesque (26!) en un problème de reconnaissance de motifs statistiques bien plus abordable.

Méthode

1. **Compter la fréquence** de chaque lettre dans le texte chiffré, généralement sous la forme d'un histogramme.
2. **Comparer** cet histogramme au profil de fréquence connu de la langue supposée du texte clair.
3. **Émettre des hypothèses de correspondance** pour les lettres les plus fréquentes du texte chiffré, en les associant aux lettres les plus fréquentes de la langue (par exemple, la lettre chiffrée la plus fréquente correspond probablement à E en français).
4. **Affiner** ces hypothèses par l'analyse des **digrammes et trigrammes** fréquents (paires et triplets de lettres consécutives) : en français, des séquences comme « ES », « LE », « DE », « ENT » ou « QUE » apparaissent très fréquemment, et leur reconnaissance dans le texte partiellement déchiffré permet de lever les ambiguïtés que la seule fréquence des lettres isolées ne résout pas.
5. **Itérer** en s'appuyant sur le sens progressivement révélé du texte partiellement déchiffré, jusqu'à obtenir un texte cohérent et complet — cette dernière étape mobilise autant l'intuition linguistique que l'analyse statistique pure.

Pour le chiffre de César en particulier, la faiblesse structurelle se double d'une faiblesse triviale d'espace de clés : il suffit de tester systématiquement les **25 décalages possibles** (une attaque exhaustive quasi instantanée, même à la main) pour identifier le bon, ce qui rend l'analyse fréquentielle presque superflue dans ce cas précis — la véritable leçon du chiffre de César est celle d'un espace de clés bien trop restreint pour toute utilisation sérieuse.

Le **chiffre de Vigenère**, présenté publiquement au XVI^e siècle et longtemps surnommé « le chiffre indéchiffrable », améliore le chiffre de César en appliquant, non plus un décalage unique, mais **un décalage différent à chaque position** du texte clair, ce décalage étant déterminé par une clé de longueur L répétée cycliquement sur toute la longueur du message. Par exemple, avec la clé « CLE » (soit les décalages 2, 11, 4), le message « BONJOUR » se chiffrerait lettre par lettre en appliquant successivement les décalages 2, 11, 4, 2, 11, 4, 2.

L'intérêt cryptographique de ce procédé est que, contrairement à une simple substitution, une même lettre du texte clair peut être chiffrée en plusieurs lettres différentes selon sa position — le chiffre de Vigenère **aplatit** artificiellement la distribution de fréquence des lettres du texte chiffré, ce qui le rend résistant à une analyse fréquentielle directe telle que décrite en section 2. Il reste cependant parfaitement cassable, en deux étapes bien distinctes.

Étape 1 — Trouver la longueur de la clé

- **Méthode de Kasiski (1863)** : elle repose sur l'observation que des séquences de lettres identiques dans le texte clair, si elles se retrouvent alignées avec la même portion de clé, produiront des séquences identiques dans le texte chiffré. En repérant les répétitions de séquences de lettres identiques (typiquement 3 lettres ou plus) dans le texte chiffré et en mesurant la **distance** (en nombre de lettres) séparant deux occurrences d'une même séquence, on obtient une valeur qui est très probablement un **multiple de la longueur de la clé L** . En calculant le PGCD (plus grand commun diviseur) de plusieurs distances observées sur différentes répétitions, on obtient une estimation fiable de L .
- **Indice de coïncidence (Friedman)** : c'est une mesure statistique de la probabilité que deux lettres tirées au hasard dans un texte donné soient identiques. Un texte rédigé en langue naturelle présente un indice de coïncidence relativement élevé, du fait de la distribution inégale des lettres ($\approx 0,0778$ en français), tandis qu'un texte parfaitement aléatoire (toutes les lettres équiprobables) présente un indice bas, proche de $1/26 \approx 0,0385$. La méthode consiste à découper le texte chiffré en L sous-suites (en prenant une lettre sur L , à partir de chaque position de départ), pour différentes valeurs candidates de L , puis à calculer l'indice de coïncidence moyen de ces sous-suites : la valeur de L qui donne des sous-suites dont l'indice se rapproche le plus de celui de la langue attendue est retenue, car chacune de ces sous-suites correspond alors à un simple décalage de César appliqué à un texte en langue naturelle.

Étape 2 — Casser chaque sous-alphabet

Une fois la longueur de clé L déterminée, le texte chiffré se décompose en L sous-suites indépendantes, chacune correspondant à un unique décalage de César constant (puisque la même lettre de la clé s'applique à toutes les lettres d'une même sous-suite). Chacune de ces sous-suites est alors cassable individuellement par la méthode d'analyse fréquentielle classique décrite en section 2, ce qui permet de retrouver lettre par lettre l'intégralité de la clé, puis de déchiffrer le message dans son ensemble.

4. Autres chiffrements classiques

- **Chiffre de Playfair** : ce chiffre, qui chiffre les lettres par paires (digrammes) à l'aide d'une grille 5×5 construite à partir d'un mot-clé, résiste mieux à une analyse fréquentielle simple portant sur les lettres isolées, car la fréquence de chaque lettre individuelle est « diluée » entre les 25 lettres possibles avec lesquelles elle peut être appariée. Il reste néanmoins vulnérable à une **analyse fréquentielle des digrammes** eux-mêmes : certains couples de lettres (« ES », « LE », « ON » en français) restent statistiquement plus fréquents que d'autres, et cette régularité, combinée aux contraintes géométriques propres à la grille de Playfair, permet une cryptanalyse structurée bien que plus laborieuse que celle d'une substitution simple.
- **Chiffre de transposition** : contrairement aux chiffres de substitution, qui remplacent chaque lettre par une autre, un chiffre de transposition se contente de **permuter l'ordre des lettres** du texte clair sans en changer aucune. Une conséquence directe, et immédiatement révélatrice, de ce principe est que la **distribution de fréquence des lettres** du texte chiffré est rigoureusement identique à celle du texte clair — seule leur position a changé. Un cryptanalyste qui observe qu'un texte chiffré présente exactement le profil de fréquence attendu de la langue clair (au lieu du profil aplati typique d'une substitution) en déduit immédiatement qu'il s'agit très probablement d'une transposition, et oriente sa recherche vers l'identification de la permutation utilisée (souvent via une analyse combinatoire des anagrammes plausibles, ou par recherche de motifs de colonnes dans le cas des transpositions en grille).

5. Ce que cette cryptanalyse historique enseigne

L'étude de ces chiffrements historiques n'a pas qu'une valeur culturelle : elle pose les fondations conceptuelles qui structurent encore aujourd'hui l'analyse des systèmes cryptographiques modernes.

- **Toute redondance statistique du texte clair qui « survit » au processus de chiffrement est, en principe, exploitable.** Ce principe général reste pleinement valable pour évaluer des schémas modernes mal conçus ou mal utilisés : c'est exactement la faiblesse exploitée contre le mode opératoire **ECB** (Electronic Codebook, voir chapitre 3 et TP3), où deux blocs de texte clair identiques produisent systématiquement deux blocs chiffrés identiques, laissant transparaître des motifs visuels dans les données chiffrées.
- **Un espace de clés trop restreint rend une attaque exhaustive triviale**, quelle que soit par ailleurs la sophistication apparente de l'algorithme. Le chiffre de César, avec ses 25 clés possibles seulement, en est l'illustration la plus simple ; ce principe motive directement les exigences modernes de taille de clé minimale (128 bits pour un chiffrement symétrique, 2048 bits ou plus pour RSA).
- Les concepts de **confusion** (rendre la relation entre la clé et le texte chiffré la plus complexe et la moins prévisible possible) et de **diffusion** (répartir l'influence de chaque bit du texte clair, et de chaque bit de la clé, sur l'ensemble du texte chiffré), formalisés bien plus tard par **Claude Shannon** dans sa théorie de la sécurité des systèmes de chiffrement (1949), trouvent leur motivation directe dans l'observation historique que l'analyse fréquentielle exploite précisément l'**absence** de ces deux propriétés. Un bon chiffrement moderne (AES, chapitre 3) est conçu explicitement pour maximiser confusion et diffusion, de sorte qu'aucune redondance statistique du texte clair ne subsiste de façon exploitable dans le texte chiffré.

À retenir

- L'analyse fréquentielle, fondée sur la distribution statistique caractéristique de chaque langue, casse systématiquement toute substitution mono-alphabétique, quelle que soit la taille apparente de l'espace de clés.
- Le chiffre de Vigenère se casse en deux temps : détermination de la longueur de clé (méthode de Kasiski ou indice de coïncidence), puis cryptanalyse fréquentielle classique de chaque sous-alphabet de César indépendant.
- Ces méthodes historiques posent les bases conceptuelles (confusion, diffusion, exploitation de redondance statistique) formalisées ensuite par Claude Shannon, qui structurent toujours l'évaluation des chiffrements modernes.

Chapitre 3 — Cryptanalyse des chiffrements symétriques modernes

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Attaque par force brute (exhaustive key search)

L'attaque la plus universelle et la plus conceptuellement simple contre tout chiffrement par bloc consiste à tester **systematiquement toutes les clés possibles** jusqu'à trouver celle qui produit un déchiffrement cohérent. Pour un chiffrement à clé de n bits, l'espace de clés contient 2^n valeurs possibles ; en moyenne, la bonne clé est trouvée après avoir testé la moitié de cet espace, soit une complexité attendue de $2^{(n-1)}$ essais.

Cette relation exponentielle entre la taille de clé et le coût de l'attaque fait de la **taille de clé** le tout premier paramètre de sécurité à examiner pour tout algorithme de chiffrement symétrique — avant même de s'interroger sur d'éventuelles faiblesses structurelles plus subtiles. Le cas de **DES** (*Data Encryption Standard*, standardisé en 1977) illustre parfaitement cette évolution : sa clé de seulement **56 bits**, jugée suffisante dans les années 1970, est aujourd'hui totalement inadaptée face à la puissance de calcul disponible — des architectures matérielles spécialisées (FPGA, ASIC dédiés) permettent de parcourir l'intégralité de l'espace de clés en quelques heures, rendant DES obsolète pour tout usage sécuritaire sérieux. C'est cette faiblesse d'espace de clés, bien plus que toute faille structurelle, qui a motivé le remplacement de DES par **AES** (*Advanced Encryption Standard*), dont les tailles de clé disponibles — 128, 192 ou 256 bits — placent la force brute très largement hors de portée de la technologie actuelle et raisonnablement prévisible : même en supposant une capacité de calcul colossale, épuiser un espace de 2^{128} clés resterait, à l'échelle de temps humaine, une entreprise sans issue.

2. Cryptanalyse différentielle

La **cryptanalyse différentielle** a été introduite publiquement par **Eli Biham et Adi Shamir** à la fin des années 1980, initialement dirigée contre DES. Il s'agit d'une attaque à texte clair choisi qui exploite la façon dont de petites différences dans le texte clair se propagent au travers des tours successifs d'un chiffrement par blocs. Le principe général se décompose ainsi :

1. **Choisir une paire de textes clairs** présentant une différence fixe et contrôlée, généralement exprimée comme un XOR précis entre les deux textes (par exemple, deux textes ne différant que sur quelques bits déterminés).
2. **Chiffrer les deux textes** avec la même clé secrète (inconnue de l'attaquant) et observer la différence entre les deux textes chiffrés obtenus.
3. Dans un chiffrement idéal (aléatoire), la distribution des différences en sortie serait uniforme. Mais pour un chiffrement réel, **certaines différences de sortie apparaissent avec une probabilité statistiquement plus élevée** que ne le voudrait le hasard pur : ce biais, appelé caractéristique différentielle, dépend de la structure interne de l'algorithme (en particulier des boîtes de substitution, ou boîtes-S) et, pour certaines valeurs, de la clé ou de sous-clés de tour utilisées.
4. En **répétant** l'observation sur un grand nombre de paires de textes clairs partageant la même différence, l'attaquant accumule des statistiques qui, une fois analysées, permettent de déduire de l'information sur des bits spécifiques de la clé (ou des sous-clés dérivées de la clé principale à chaque tour), réduisant ainsi l'espace de recherche nécessaire pour retrouver la clé complète.

Un point historique notable est que **DES s'est révélé partiellement vulnérable** à cette technique lorsqu'elle a été rendue publique, mais avec une résistance meilleure que celle d'une conception aléatoire : les concepteurs de DES, au sein d'IBM, avaient déjà pris en compte et limité cette catégorie d'attaque dans le choix précis des boîtes-S de l'algorithme, plusieurs années avant que la cryptanalyse différentielle ne soit publiée académiquement. Il est aujourd'hui admis que la NSA (*National Security Agency*) avait connaissance de cette technique d'attaque avant sa découverte indépendante par la communauté académique, ce qui illustre une constante en cryptographie : une technique d'attaque peut être connue en secret par certains acteurs bien avant sa publication ouverte, d'où l'importance du principe de conception défensive et de marge de sécurité évoqué au chapitre 1.

3. Cryptanalyse linéaire

La **cryptanalyse linéaire**, introduite par **Mitsuru Matsui** en 1993, constitue une autre grande famille d'attaques structurelles contre les chiffrements par blocs, cette fois fondée sur une attaque à texte clair connu (moins exigeante pour l'attaquant que l'attaque à texte clair choisi requise pour la cryptanalyse différentielle).

Le principe consiste à rechercher des **approximations linéaires** : des équations combinant, par un XOR, certains bits du texte clair, certains bits du texte chiffré, et certains bits de la clé, et qui se trouvent être vraies avec une probabilité sensiblement différente de $1/2$ (la probabilité qu'on attendrait pour une relation purement aléatoire). Une telle approximation, même faiblement biaisée, révèle une corrélation exploitable entre des bits d'entrée, de sortie et de clé.

En combinant une approximation linéaire suffisamment biaisée avec un très grand nombre de couples (texte clair, texte chiffré) connus, un attaquant peut retrouver des bits spécifiques de la clé par un mécanisme de **vote statistique** : pour chaque hypothèse possible sur les bits de clé recherchés, on compte combien de couples observés vérifient l'approximation linéaire, et l'hypothèse qui s'écarte le plus de la probabilité $1/2$ attendue est retenue comme la plus probable. La combinaison de plusieurs approximations linéaires à travers les différents tours de l'algorithme repose sur le **lemme du pilage** (*piling-up lemma*), qui permet de calculer le biais résultant d'une chaîne d'approximations élémentaires combinées.

4. Pourquoi AES résiste-t-il mieux ?

AES (choisi en 2001 à l'issue d'un concours public international organisé par le NIST américain, sous le nom original de *Rijndael*, proposé par les cryptographes belges Joan Daemen et Vincent Rijmen) a été conçu, dès l'origine, en intégrant explicitement des **critères de résistance** aux attaques différentielles et linéaires — contrairement à DES, dont la résistance à ces attaques n'a été qu'une conséquence partielle et empirique de choix de conception antérieurs à la découverte publique de ces techniques.

Concrètement, la conception d'AES repose sur :

- une **structure des boîtes-S** mathématiquement optimisée (fondée sur l'inverse dans un corps fini, $GF(2^8)$) pour minimiser les biais différentiels et linéaires exploitables ;
- un **nombre de tours** (10, 12 ou 14 selon la taille de clé) dimensionné avec une **marge de sécurité** volontairement confortable au-delà du minimum théoriquement nécessaire pour résister aux attaques connues au moment de sa conception, anticipant ainsi de futures améliorations éventuelles des techniques cryptanalytiques.

Cette démarche de conception, fondée sur des **preuves de résistance** (bornes démontrables sur la probabilité maximale des caractéristiques différentielles et linéaires) plutôt que sur une simple absence d'attaque connue au moment de la publication, explique pourquoi, à ce jour, **aucune attaque académique publiée ne casse AES pleine puissance** plus rapidement qu'une recherche exhaustive proche de l'optimum théorique de $2^{(n-1)}$ — un résultat remarquable pour un algorithme utilisé à si grande échelle depuis plus de vingt ans.

5. Attaques structurelles sur les modes opératoires

Un algorithme de chiffrement par blocs intrinsèquement robuste comme AES peut néanmoins être compromis en pratique par un **mode opératoire** mal choisi ou mal utilisé — un mode opératoire définissant la façon dont l'algorithme de bloc, qui ne chiffre qu'un bloc de taille fixe (128 bits pour AES), est appliqué à un message de longueur arbitraire.

Mode	Faiblesse exploitable
ECB (Electronic Codebook)	chaque bloc de texte clair est chiffré indépendamment avec la même clé, sans aucun élément aléatoire ; des blocs de texte clair identiques produisent donc systématiquement des blocs chiffrés identiques, ce qui laisse transparaître des motifs structurels du message d'origine (illustré classiquement par le chiffrement ECB d'une image, où les zones uniformes restent visuellement reconnaissables — voir TP3)
CBC sans authentification	chaque bloc de texte clair est combiné par XOR avec le bloc chiffré précédent avant chiffrement, ce qui élimine la faiblesse d'ECB, mais un système qui révèle si le padding (remplissage, souvent PKCS#7) du texte déchiffré est valide ou non ouvre la voie à une attaque par oracle de padding , permettant de déchiffrer progressivement un message entier sans connaître la clé (détaillée au chapitre 6 et en TP3)
Réutilisation de nonce/IV (modes CTR, GCM)	ces modes transforment un chiffrement par blocs en un chiffrement de flux, en combinant par XOR le texte clair avec un flux de clé généré à partir d'un nonce ou vecteur d'initialisation (IV) unique ; si ce nonce est accidentellement réutilisé avec la même clé pour deux messages différents, un attaquant qui intercepte les deux textes chiffrés peut calculer le XOR des deux textes clairs correspondants (car le flux de clé identique s'annule), ce qui annule une grande partie de la sécurité recherchée et peut, combiné à des techniques d'analyse de langage naturel, permettre de retrouver les deux messages

Il est essentiel de souligner que ces trois faiblesses **ne remettent aucunement en cause la sécurité de l'algorithme de bloc lui-même** (AES reste, dans les trois cas, parfaitement robuste) : elles résultent exclusivement d'un mauvais usage du mode opératoire qui l'entoure. Cette distinction souligne un principe central de la cryptographie appliquée : il est très fortement déconseillé de composer soi-même un algorithme de bloc et un mode opératoire de façon artisanale ; il est bien préférable de s'appuyer sur des **schémas authentifiés éprouvés et standardisés**, tels qu'AES-GCM ou ChaCha20-Poly1305, qui associent chiffrement et vérification d'intégrité/authenticité de façon à prévenir nativement ces classes de faiblesses.

6. Attaques par compromis temps-mémoire

Une autre famille de techniques cryptanalytiques, transversale à plusieurs contextes, repose sur l'idée d'un **compromis entre temps de calcul et espace de stockage** pour inverser une fonction à sens unique. Les **tables arc-en-ciel** (*rainbow tables*) en sont l'illustration la plus connue : au lieu de recalculer, pour chaque tentative, l'application complète de la fonction à inverser (ce qui coûterait beaucoup de temps de calcul mais peu de mémoire), ou de précalculer et stocker l'intégralité de toutes les correspondances possibles (ce qui coûterait beaucoup de mémoire mais permettrait une recherche instantanée), une table arc-en-ciel construit une structure intermédiaire, fondée sur des chaînes de réduction et de calcul alternées, qui réduit considérablement l'espace de stockage nécessaire au prix d'un temps de recherche légèrement accru par rapport à une table de correspondance complète.

Ces tables ont été utilisées historiquement, avec un succès notable, contre des **hachages de mots de passe non salés** — un sujet approfondi au chapitre 4, où l'on verra que le **salage** (l'ajout d'une valeur aléatoire unique à chaque mot de passe avant hachage) neutralise précisément ce type d'attaque en rendant impossible la réutilisation d'une table précalculée d'un utilisateur à l'autre.

À retenir

- La taille de clé fixe la borne théorique de résistance à la force brute ($2^{(n-1)}$ essais en moyenne) ; DES (56 bits) est aujourd'hui cassable en pratique, AES (128 bits ou plus) ne l'est pas avec la technologie actuelle et prévisible.
- Cryptanalyse différentielle (Biham-Shamir) et cryptanalyse linéaire (Matsui) exploitent des biais statistiques internes à la structure de l'algorithme et ont directement influencé la conception défensive d'AES.
- La grande majorité des compromissions pratiques de chiffrements symétriques modernes viennent d'un mauvais mode opératoire ou d'une mauvaise gestion des nonces/IV, pas d'une faille dans l'algorithme de bloc lui-même — d'où l'importance de privilégier des schémas authentifiés standardisés (AES-GCM, ChaCha20-Poly1305).

Chapitre 4 — Attaques sur les fonctions de hachage

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Rappel des propriétés attendues

Une fonction de hachage cryptographique transforme une entrée de taille arbitraire en une empreinte (ou condensat, *digest*) de taille fixe, et doit garantir trois propriétés de sécurité complémentaires, chacune correspondant à un scénario d'attaque distinct :

- **La résistance à la préimage** : étant donné une empreinte h , il doit être calculatoirement impossible de retrouver un message m tel que $H(m) = h$. Cette propriété protège, par exemple, le stockage d'un mot de passe sous forme de son empreinte : un attaquant qui vole la base de données d'empreintes ne doit pas pouvoir en déduire directement les mots de passe d'origine.
- **La résistance à la seconde préimage** : étant donné un message m_1 spécifique, il doit être calculatoirement impossible de trouver un second message m_2 , distinct de m_1 , tel que $H(m_1) = H(m_2)$. Cette propriété protège contre la substitution d'un document par un autre document ayant la même empreinte, dans un contexte où le document original m_1 est connu et fixé à l'avance.
- **La résistance aux collisions** : il doit être calculatoirement impossible de trouver **deux messages distincts quelconques** m_1 et m_2 (sans qu'aucun des deux ne soit fixé à l'avance) tels que $H(m_1) = H(m_2)$. C'est la propriété la plus faible des trois à garantir en théorie (l'attaquant a plus de liberté), et c'est donc, en pratique, la première à céder lorsqu'une fonction de hachage commence à montrer des faiblesses — comme l'illustre l'histoire de MD5 et SHA-1 (section 3).

Ces trois propriétés sont hiérarchisées : une fonction résistante aux collisions n'est pas automatiquement résistante à la préimage (ce sont des propriétés mathématiquement distinctes), mais en pratique, une attaque en collision découverte sur une fonction constitue toujours un signal d'alerte fort sur sa robustesse globale et précède souvent la découverte d'attaques plus fortes.

2. Le paradoxe des anniversaires et l'attaque par collision

Pour une empreinte de n bits, une recherche naïve de collision par force brute — en calculant des empreintes de messages au hasard et en les comparant une à une à une empreinte cible fixée — coûterait en théorie jusqu'à 2^n essais dans le pire cas. Mais si l'objectif est simplement de trouver **une** collision quelconque entre deux messages, sans viser une empreinte cible précise, le coût réel est bien plus faible : c'est ce que révèle le **paradoxe des anniversaires**.

Ce paradoxe, purement probabiliste, s'énonce classiquement ainsi : dans un groupe de seulement 23 personnes choisies au hasard, la probabilité qu'au moins deux d'entre elles partagent la même date d'anniversaire dépasse déjà 50 %, un résultat qui surprend l'intuition (on s'attendrait naïvement à avoir besoin de proche de 365 personnes). La raison mathématique est que le nombre de **paires** possibles parmi k éléments croît en $k(k-1)/2$, c'est-à-dire de façon quadratique et non linéaire : il ne faut donc pas comparer chaque nouvel élément à une seule cible, mais chaque nouvel élément à tous les éléments précédemment générés.

Appliqué au hachage, ce même raisonnement montre qu'il suffit de générer environ **$2^{(n/2)}$ messages** (et de comparer leurs empreintes entre elles, deux à deux) pour obtenir, avec une probabilité significative, une collision — un coût très inférieur aux 2^n essais qu'on pourrait naïvement attendre. La conséquence directe et fondamentale de ce résultat est qu'une **fonction de hachage de n bits de sortie n'offre, en réalité, qu'une sécurité de $n/2$ bits contre les collisions** (bien qu'elle conserve, elle, une sécurité de n bits contre la préimage et la seconde préimage, des attaques qui restent ciblées sur un message précis). Cette division par deux explique pourquoi **MD5**, avec ses 128 bits de sortie, n'offre en réalité qu'environ 64 bits de sécurité en collision — un niveau aujourd'hui totalement insuffisant et concrètement atteint — et pourquoi **SHA-1**, avec ses 160 bits de sortie (soit environ 80 bits de sécurité théorique en collision), a été déconseillé après la démonstration d'une collision pratique en 2017 (l'attaque publiquement documentée « SHAttered »), qui a exploité des faiblesses structurelles de l'algorithme permettant d'atteindre une collision à un coût sensiblement inférieur même à cette borne théorique de 2^{80} .

3. Attaques historiques marquantes

L'histoire récente des fonctions de hachage cryptographiques illustre concrètement la dynamique évoquée au chapitre 1 : un algorithme jugé sûr à sa conception peut être progressivement fragilisé, puis définitivement cassé, à mesure que la cryptanalyse publique progresse.

- **MD5** (conçu en 1991) : des collisions pratiques ont été démontrées dès 2004-2005 par des équipes de recherche, exploitant des faiblesses structurelles dans la fonction de compression interne de l'algorithme. Ces techniques ont ensuite été affinées jusqu'à permettre la construction de collisions « à préfixe choisi » (permettant d'insérer une collision dans un contexte préparé par l'attaquant), exploitées notamment dans la conception du logiciel malveillant Flame (documenté publiquement en 2012), qui a utilisé une collision MD5 pour forger un faux certificat numérique paraissant signé par Microsoft.
- **SHA-1** (conçu en 1995) : une collision pratique complète a été démontrée par une équipe de chercheurs de Google et du CWI (Centrum Wiskunde & Informatica, Pays-Bas) en 2017, sous le nom d'attaque « SHAttered », en produisant deux fichiers PDF distincts partageant la même empreinte SHA-1. Cette démonstration publique a fortement accéléré la migration généralisée des systèmes encore dépendants de SHA-1 (certificats numériques, systèmes de contrôle de version comme Git dans certains usages) vers des fonctions plus robustes.
- **SHA-2 (incluant SHA-256, SHA-512) et SHA-3** : à ce jour, aucune attaque pratique publiée ne remet en cause la sécurité de ces familles de fonctions ; elles restent les standards recommandés pour tout usage cryptographique exigeant une résistance aux collisions et à la préimage. SHA-3, standardisé en 2015 à l'issue d'un concours public organisé par le NIST (à l'image du concours ayant abouti à AES), repose sur une construction interne (*sponge construction*, ou construction en éponge) structurellement différente de celle de SHA-2, ce qui offre une diversité de conception rassurante en cas de découverte future d'une faiblesse propre à une famille de construction donnée.

4. Attaque par extension de longueur (length extension attack)

Les fonctions de hachage fondées sur la **construction de Merkle-Damgård** — c'est le cas de MD5, SHA-1 et SHA-2 — traitent le message d'entrée par blocs successifs, chaque bloc étant combiné à un état interne mis à jour progressivement, l'empreinte finale étant simplement l'état interne obtenu après traitement du dernier bloc. Cette construction, bien que suffisante pour garantir la résistance aux collisions dans certaines conditions, présente une propriété structurelle exploitable : l'**attaque par extension de longueur**.

Le principe est le suivant : connaissant uniquement $H(\text{message})$ (l'empreinte) et la **longueur** du message d'origine (mais sans connaître le contenu exact de `message`), un attaquant peut calculer directement $H(\text{message} || \text{padding} || \text{extension})$ pour une extension de son choix qu'il ajoute à la suite — **sans jamais avoir eu connaissance du contenu du message original**. Ceci est possible car l'état interne de la fonction après traitement de `message` est exactement égal à $H(\text{message})$ lui-même : l'attaquant peut donc reprendre le calcul de hachage exactement là où il s'était arrêté, comme s'il connaissait le message, en repartant simplement de l'empreinte connue.

La conséquence pratique la plus importante de cette propriété concerne l'authentification de messages : construire un code d'authentification de message (MAC) par simple concaténation, sous la forme $H(\text{clé} || \text{message})$, est **dangereux**, car un attaquant qui observe $H(\text{clé} || \text{message})$ et la longueur du message (mais pas la clé) peut forger, grâce à l'extension de longueur, une empreinte valide pour `clé || message || padding || extension`, c'est-à-dire produire un MAC valide pour un message étendu de son choix, sans jamais avoir connu la clé secrète. C'est précisément pour neutraliser cette faille structurelle que la construction **HMAC** (*Hash-based Message Authentication Code*, étudiée en détail au module *Cryptographie*) a été spécifiquement conçue : en imbriquant la clé de part et d'autre du calcul de hachage selon un schéma précis, HMAC rend l'attaque par extension de longueur inopérante, quelle que soit la fonction de hachage sous-jacente utilisée.

5. Hachage de mots de passe : attaques et contre-mesures

Le hachage de mots de passe constitue un cas d'usage particulier des fonctions de hachage, où les propriétés recherchées diffèrent sensiblement de celles requises pour, par exemple, vérifier l'intégrité d'un fichier téléchargé. Plusieurs classes d'attaques s'exercent spécifiquement contre ce cas d'usage, chacune appelant une contre-mesure dédiée.

Attaque	Principe	Contre-mesure
Force brute / attaque par dictionnaire	tester systématiquement des mots de passe candidats (issus d'un dictionnaire de mots de passe courants, ou générés exhaustivement) et comparer l'empreinte obtenue à celle stockée	politique de mots de passe suffisamment robustes (longueur, diversité), limitation du nombre de tentatives côté serveur, journalisation des essais infructueux
Table arc-en-ciel	utilisation de tables précalculées reliant des empreintes courantes aux mots de passe correspondants, permettant une inversion quasi instantanée d'un hachage non salé connu	salage : ajout d'une valeur aléatoire unique (le sel), différente pour chaque utilisateur, combinée au mot de passe avant hachage ; une table précalculée pour un sel donné devient inutile pour tout autre utilisateur ayant un sel différent
Calcul massivement parallèle (GPU/ASIC)	les fonctions de hachage génériques (SHA-256 par exemple) sont conçues pour être rapides à calculer (un avantage pour la vérification d'intégrité de fichiers, mais un inconvénient direct ici), ce qui permet à un attaquant disposant de matériel de calcul parallèle de tester des milliards de mots de passe candidats par seconde	recours à des fonctions spécialement conçues pour être lentes et coûteuses en ressources : bcrypt, scrypt, ou Argon2 (vainqueur de la compétition <i>Password Hashing Competition</i>), qui intègrent un facteur de coût réglable , permettant d'augmenter délibérément le temps et/ou la mémoire nécessaires à chaque calcul, y compris pour un attaquant disposant de matériel spécialisé

Le point commun de ces trois contre-mesures est qu'elles ne modifient en rien la sécurité mathématique intrinsèque de la fonction de hachage sous-jacente (en termes de résistance aux collisions ou à la préimage) : elles visent spécifiquement à **augmenter le coût pratique** d'une attaque exhaustive contre un espace de mots de passe réalistes, qui est structurellement bien plus restreint que l'espace complet des messages possibles pour lequel les fonctions de hachage génériques ont été conçues.

6. Méthodologie d'évaluation d'une fonction de hachage

Face à une fonction de hachage donnée, dans un contexte d'usage particulier, une démarche d'évaluation rigoureuse suit les étapes suivantes :

1. **Vérifier la taille de sortie** de la fonction, qui détermine la borne théorique de sécurité en collision (rappel : sécurité en collision = taille de sortie divisée par deux, d'après le paradoxe des anniversaires).
2. **Vérifier l'absence d'attaques publiées** de préimage ou de collision pratiques contre la fonction considérée, en consultant l'état de l'art académique — une fonction jugée sûre aujourd'hui doit continuer d'être suivie dans le temps (principe de sécurité évolutive, chapitre 1).
3. **Vérifier l'adéquation au contexte d'usage précis** : une fonction parfaitement sûre pour l'intégrité de fichiers ou la construction de signatures numériques (SHA-256) n'est **pas nécessairement adaptée**, sans adaptation supplémentaire, au hachage de mots de passe — sa rapidité de calcul, qui est un avantage dans le premier contexte, devient une faiblesse dans le second, d'où la nécessité d'un facteur de travail réglable et d'un salage systématique (section 5).

À retenir

- Le paradoxe des anniversaires divise par deux, en bits, la sécurité effective d'une fonction de hachage contre les collisions par rapport à sa sécurité contre la préimage.
- MD5 et SHA-1 sont aujourd'hui cassés en pratique pour les collisions (attaques SHAttered pour SHA-1, exploitations documentées comme Flame pour MD5) et ne doivent plus être utilisés pour un usage sécuritaire ; SHA-2 et SHA-3 restent les standards recommandés.
- La construction Merkle-Damgård (MD5, SHA-1, SHA-2) est vulnérable à l'attaque par extension de longueur, ce qui rend dangereuse la construction naïve d'un MAC par simple concaténation $H(\text{clé} || \text{message})$ — HMAC neutralise spécifiquement cette faiblesse.
- Le hachage de mots de passe exige des fonctions dédiées, volontairement lentes et salées (bcrypt, scrypt, Argon2) — jamais une fonction générique rapide (SHA-256 brut) utilisée seule.

Chapitre 5 — Cryptanalyse des systèmes asymétriques

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. RSA : rappel du principe et hypothèse de sécurité

La sécurité de **RSA** repose entièrement sur la difficulté supposée de **factoriser un grand nombre entier** n , produit de deux grands nombres premiers distincts p et q ($n = p \times q$), choisis secrètement lors de la génération de la paire de clés. Connaître n seul (une information publique, incluse dans la clé publique) ne permet pas, en pratique, de retrouver p et q si ces derniers sont suffisamment grands et choisis avec soin — c'est précisément cette asymétrie de difficulté (facile de multiplier deux nombres premiers, difficile de retrouver les facteurs à partir du produit) qui fonde la sécurité de tout le système.

Il est essentiel de comprendre la nature de cette garantie : **aucun algorithme classique connu à ce jour ne factorise efficacement**, c'est-à-dire en temps polynomial par rapport à la taille du nombre à factoriser, un entier suffisamment grand. Mais il s'agit d'une **hypothèse calculatoire empirique**, fondée sur l'échec collectif de décennies de recherche à trouver un tel algorithme efficace, et non d'une **preuve mathématique absolue** d'impossibilité. Cette distinction est cruciale : elle explique pourquoi la sécurité de RSA doit être réévaluée en permanence à la lumière des progrès algorithmiques (section 2) et technologiques (menace quantique, également section 2), conformément au principe de sécurité relative et évolutive introduit au chapitre 1.

2. Attaques par factorisation

Plusieurs approches, de puissance très inégale, permettent en théorie d'attaquer directement le problème mathématique de factorisation sous-jacent à RSA :

- **Force brute / essai de division systématique** : consiste à tester la divisibilité de n par tous les nombres premiers possibles jusqu'à sa racine carrée. Cette approche, dont la complexité croît de façon quasi exponentielle avec la taille de n , est totalement impraticable dès que n dépasse quelques dizaines de chiffres décimaux — a fortiori pour des clés RSA de taille réaliste (2048 bits, soit plus de 600 chiffres décimaux).
- **Algorithmes sous-exponentiels** : le **General Number Field Sieve** (GNFS, ou « crible général sur les corps de nombres ») est, à ce jour, l'algorithme classique le plus efficace connu pour factoriser de grands entiers génériques. Sa complexité, bien que **sous-exponentielle** (donc bien meilleure que la force brute, mais toujours loin d'être polynomiale), reste hors de portée pratique pour des clés RSA de 2048 bits avec la technologie de calcul actuelle. L'évolution historique des records de factorisation illustre concrètement cette progression et justifie l'augmentation progressive de la taille de clé recommandée dans le temps : une clé de 512 bits a été factorisée dès 1999, une clé de 768 bits en 2009, une clé de 1024 bits est aujourd'hui considérée comme fragilisée et déconseillée pour tout nouvel usage, tandis qu'une clé de **2048 bits reste recommandée** comme minimum pour un usage général aujourd'hui, avec 3072 ou 4096 bits pour des besoins de sécurité à plus long terme.
- **Menace quantique (algorithme de Shor)** : l'**algorithme de Shor**, conçu théoriquement en 1994 pour un ordinateur quantique, factoriserait un grand nombre entier en **temps polynomial** — une rupture radicale par rapport à tous les algorithmes classiques connus. Si un ordinateur quantique suffisamment puissant et suffisamment stable (à grande échelle, avec correction d'erreurs) devenait un jour disponible, il casserait non seulement RSA mais également tout système fondé sur le problème du logarithme discret (Diffie-Hellman, ECC classiques, section 4 et 5). Cette menace, encore hypothétique aujourd'hui mais prise très au sérieux par la communauté cryptographique, motive dès à présent la standardisation d'algorithmes dits **post-quantiques**, conçus pour résister à un adversaire disposant d'un ordinateur quantique (exemples issus du processus de standardisation du NIST : ML-KEM, anciennement Kyber, pour l'échange de clés, et ML-DSA, anciennement Dilithium, pour la signature numérique).

3. Attaques exploitant une mauvaise implémentation de RSA

Il est crucial de distinguer les attaques précédentes, qui s'attaquent directement au problème mathématique de la factorisation, des attaques suivantes, qui **n'exploitent aucune faiblesse du problème mathématique sous-jacent**, mais des erreurs pratiques commises lors de la génération des clés ou de l'utilisation de l'algorithme — des erreurs, hélas, **bien plus fréquentes en réalité** que toute avancée en factorisation.

Attaque	Condition d'application
Exposant public e trop petit sans padding adéquat (typiquement $e = 3$)	si un message est suffisamment court et chiffré sans un schéma de padding correct (comme OAEP), le texte chiffré peut être proche d'un simple cube du message, ce qui permet à un attaquant d'en extraire une racine cubique directe et de retrouver le message clair sans jamais attaquer la factorisation
Modules partagés / génération de clés faible	si deux paires de clés RSA distinctes, générées par exemple avec un générateur aléatoire de mauvaise qualité, partagent accidentellement un même facteur premier (par exemple p commun à deux modules n_1 et n_2 différents), alors le simple calcul du PGCD des deux modules révèle instantanément ce facteur premier commun, effondrant la sécurité des deux clés simultanément — un cas documenté publiquement à grande échelle dans des études académiques sur des certificats RSA collectés sur Internet, révélant que la génération d'aléa de mauvaise qualité sur certains équipements avait produit des modules partagés entre appareils distincts
Attaque de Wiener	si l'exposant privé d est choisi trop petit par rapport à la taille du module n (une pratique parfois adoptée, à tort, pour accélérer les calculs de déchiffrement), la théorie des fractions continues permet de retrouver d directement à partir de la seule clé publique, sans passer par la factorisation de n
Attaque de Coppersmith	dans certaines conditions particulières où l'attaquant connaît une partie substantielle des bits de p , de q , ou du message clair (par exemple à cause d'une fuite d'information partielle ailleurs dans le système), des techniques fondées sur la réduction de réseaux (résultat de Coppersmith, 1996) permettent de compléter l'information manquante bien plus efficacement qu'une recherche exhaustive
Réutilisation de nonce / mauvaise génération aléatoire	un générateur pseudo-aléatoire de mauvaise qualité utilisé lors de la génération des nombres premiers p et q peut produire des clés prévisibles ou corrélées entre plusieurs appareils — un cas réel documenté a concerné des équipements embarqués bon marché dont le processus de démarrage offrait une entropie insuffisante au moment de la génération des clés

Cette liste illustre un principe déjà rencontré au chapitre 1 : dans la très grande majorité des compromissions réelles de systèmes RSA, la faiblesse exploitée réside dans **l'implémentation et la génération des paramètres**, et non dans une percée sur le problème mathématique de factorisation lui-même, qui demeure, pour des paramètres correctement choisis, hors de portée de toute attaque connue.

4. Diffie-Hellman et logarithme discret

La sécurité du protocole d'échange de clés **Diffie-Hellman** (ainsi que du schéma de chiffrement **ElGamal**, fondé sur le même principe mathématique) repose sur la difficulté supposée du **problème du logarithme discret** : étant donné un générateur g , un module premier p , et la valeur $g^x \bmod p$, il est supposé calculatoirement difficile de retrouver l'exposant secret x . Comme pour la factorisation dans le cas de RSA, il s'agit d'une hypothèse calculatoire, appuyée sur l'absence d'algorithme classique efficace connu, et non d'une preuve d'impossibilité.

Les meilleures attaques classiques connues contre ce problème, regroupées sous le nom de méthodes **index calculus**, présentent une complexité comparable à celle du GNFS utilisé pour la factorisation RSA, à taille de groupe équivalente — ce qui explique pourquoi les recommandations de taille de paramètres pour Diffie-Hellman (taille du module p) suivent des ordres de grandeur similaires à celles de RSA (2048 bits recommandés aujourd'hui).

Attaque pratique notable — Logjam (2015) : cette attaque, documentée publiquement, a révélé qu'un très grand nombre de serveurs déployés sur Internet réutilisaient, par commodité d'implémentation, les **mêmes groupes premiers standardisés** de taille relativement modeste (512 bits notamment) pour leurs échanges Diffie-Hellman. Cette réutilisation massive d'un même paramètre partagé a permis à des chercheurs de réaliser, une seule fois pour toutes, un **précalcul coûteux mais mutualisable** contre ce groupe précis, rendant ensuite l'attaque de logarithme discret praticable en quelques minutes contre n'importe quelle connexion utilisant ce même groupe de 512 bits. Cette attaque illustre un principe cryptanalytique important et transversal : un paramètre cryptographique partagé et sous-dimensionné affaiblit simultanément **tous** les systèmes qui le réutilisent, car le coût d'attaque, bien qu'élevé, n'a besoin d'être payé qu'une seule fois pour compromettre un nombre arbitrairement grand de cibles ultérieures.

5. Courbes elliptiques (ECC)

La **cryptographie sur courbes elliptiques** (ECC) repose sur une variante du problème du logarithme discret, transposée dans le groupe des points d'une courbe elliptique définie sur un corps fini, plutôt que dans le groupe multiplicatif des entiers modulo un nombre premier utilisé par Diffie-Hellman classique. Ce problème, appelé **ECDLP** (*Elliptic Curve Discrete Logarithm Problem*), est considéré comme substantiellement **plus difficile**, à taille de clé égale, que son équivalent sur les entiers : les meilleures attaques classiques connues contre ECDLP (essentiellement des variantes de l'algorithme de Pollard, de complexité pleinement exponentielle) ne bénéficient pas des raccourcis sous-exponentiels dont profitent le GNFS contre RSA ou l'index calculus contre Diffie-Hellman classique.

Cette différence structurelle a une conséquence pratique directe et significative : une clé ECC de **256 bits** offre un niveau de sécurité comparable à une clé RSA de **3072 bits**, pour un coût de calcul très inférieur et une taille de clé bien plus compacte. Cet avantage explique l'adoption croissante d'ECC dans les protocoles cryptographiques modernes — on retrouve notamment **ECDSA** pour la signature numérique et **X25519** pour l'échange de clés dans des standards largement déployés comme TLS 1.3 ou SSH, où la compacité des clés et l'efficacité des calculs sont des critères particulièrement appréciés, en plus du niveau de sécurité offert.

6. Démarche générale de cryptanalyse d'un système asymétrique

Face à un système cryptographique asymétrique donné, qu'il soit fondé sur RSA, Diffie-Hellman ou ECC, une démarche d'évaluation structurée suit les étapes suivantes, ordonnées par ordre de probabilité décroissante de succès pour un attaquant réaliste :

1. **Vérifier les paramètres utilisés** : la taille de clé est-elle conforme aux recommandations actuelles (2048 bits ou plus pour RSA, 256 bits ou plus pour ECC) ? La courbe elliptique ou le groupe utilisé est-il un paramètre **standardisé et publiquement audité**, ou un paramètre « maison » non vérifié par la communauté, ce qui constituerait en soi un signal d'alerte majeur (rappel du principe de Kerckhoffs, chapitre 1) ?
2. **Rechercher des erreurs de génération de clé** : facteurs premiers accidentellement partagés entre plusieurs clés, aléa insuffisant lors de la génération, réutilisation de paramètres entre systèmes distincts (comme dans l'attaque Logjam).
3. **Vérifier l'usage correct du padding approprié** : le schéma **OAEP** (*Optimal Asymmetric Encryption Padding*) pour le chiffrement RSA, et le schéma **PSS** (*Probabilistic Signature Scheme*) pour la signature RSA, sont nécessaires pour prévenir des classes entières d'attaques structurelles contre une utilisation « manuelle » et naïve de RSA (comme celles exploitant un petit exposant public, section 3).
4. **Ne considérer l'attaque directe du problème mathématique** (factorisation « brute » de RSA, ou logarithme discret « brut ») **que comme un dernier recours théorique** dans cette démarche d'évaluation : en pratique, l'immense majorité des compromissions documentées de systèmes asymétriques proviennent des trois points précédents, tandis que le problème mathématique fondamental, pour des paramètres correctement choisis et à jour, demeure hors de portée de toute attaque classique connue.

À retenir

- La sécurité de RSA et de Diffie-Hellman repose sur des hypothèses calculatoires (difficulté de la factorisation, difficulté du logarithme discret), appuyées sur l'absence d'algorithme classique efficace connu — pas sur une preuve d'impossibilité mathématique absolue.
- Les attaques réellement exploitées en pratique ciblent presque toujours une mauvaise implémentation (aléa faible, paramètres partagés ou sous-dimensionnés, padding absent ou incorrect) plutôt que l'algorithme mathématique lui-même, qui reste robuste pour des paramètres correctement choisis.
- ECC offre, à taille de clé égale, une sécurité substantiellement supérieure à RSA classique (256 bits ECC $\approx$ 3072 bits RSA), ce qui explique son adoption croissante dans les protocoles modernes.
- La menace de l'informatique quantique (algorithme de Shor, qui casserait en temps polynomial RSA, Diffie-Hellman et ECC classiques) motive dès aujourd'hui la transition en cours vers la cryptographie post-quantique standardisée par le NIST.

Chapitre 6 — Attaques par canaux auxiliaires et attaques pratiques

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Principe des attaques par canaux auxiliaires (side-channel)

Une attaque par **canal auxiliaire** (*side-channel*) se distingue fondamentalement de toutes les attaques étudiées dans les chapitres précédents : elle n'exploite **aucune faiblesse mathématique** de l'algorithme cryptographique lui-même, aussi robuste soit-il en théorie, mais une **information physique involontairement fuitée par son implémentation concrète**. Cette information peut prendre des formes très diverses : le temps d'exécution d'une opération, la consommation électrique instantanée du circuit qui l'exécute, le rayonnement électromagnétique émis, le comportement observable du cache du processeur, ou encore le contenu précis des messages d'erreur renvoyés par un système.

Ces attaques constituent un rappel fondamental, déjà annoncé au chapitre 1 : la sécurité effective d'un système cryptographique dépend **autant de la qualité de son implémentation que de la robustesse mathématique de sa conception**. Un algorithme parfaitement sûr sur le papier — disposant même d'une preuve de sécurité formelle — peut être totalement compromis en pratique si son implémentation laisse fuiter, même très indirectement, de l'information sur les secrets manipulés. C'est un domaine de recherche à l'intersection de la cryptographie théorique, de l'architecture matérielle et de l'ingénierie logicielle.

2. Attaques temporelles (timing attacks)

Le principe d'une attaque temporelle repose sur l'observation suivante : si le **temps d'exécution** d'une opération cryptographique dépend, même de façon subtile, des bits secrets qu'elle manipule, alors un attaquant capable de mesurer précisément ce temps peut en déduire de l'information sur le secret, parfois octet par octet.

Un exemple pédagogique classique concerne une **comparaison de mot de passe (ou de jeton secret) mal implémentée**, qui compare les caractères un par un et s'arrête dès qu'une différence est détectée (comportement naturel d'une comparaison de chaînes de caractères « naïve », comme celle offerte par défaut dans de nombreux langages de programmation). Dans ce cas, une comparaison s'arrêtant plus tôt (parce que le premier caractère testé est déjà incorrect) prend statistiquement moins de temps qu'une comparaison s'arrêtant plus tard (parce que les premiers caractères testés étaient corrects) : en mesurant ces variations de temps, même infimes, sur un très grand nombre d'essais successifs, un attaquant peut en théorie reconstituer le secret comparé, position par position, sans jamais avoir eu à le deviner d'un seul coup.

Contre-mesure : la protection standard contre cette classe d'attaque consiste à utiliser des **comparaisons en temps constant** (*constant-time comparison*), c'est-à-dire des fonctions de comparaison conçues pour parcourir systématiquement l'intégralité de la donnée comparée, quel que soit le résultat intermédiaire obtenu à chaque étape, de sorte que le temps d'exécution total ne dépende jamais du contenu du secret. La plupart des bibliothèques cryptographiques modernes fournissent une fonction dédiée à cet effet, qu'il convient d'utiliser systématiquement pour toute comparaison portant sur une donnée secrète (clé, jeton d'authentification, code d'authentification de message), plutôt que de recourir à une comparaison générique du langage utilisé.

3. Attaques par oracle de padding (padding oracle)

Cette classe d'attaque, décrite en détail en TP3, illustre de façon particulièrement concrète comment une information apparemment anodine — la simple validité d'un padding — peut suffire à compromettre entièrement la confidentialité d'un message chiffré, sans jamais avoir besoin de connaître la clé de chiffrement.

Le principe repose sur le mode opératoire **CBC** (*Cipher Block Chaining*) combiné à un schéma de remplissage (padding) tel que **PKCS#7**, utilisé pour compléter le dernier bloc d'un message dont la longueur n'est pas un multiple exact de la taille de bloc. Si un système déchiffre un texte chiffré et **révèle**, d'une façon ou d'une autre — que ce soit par un code d'erreur explicitement différent selon que le padding est valide ou non, ou même simplement par une différence de temps de réponse mesurable entre les deux cas — cette information de validité du padding constitue un « oracle » que l'attaquant peut interroger de façon répétée et systématique. En modifiant méthodiquement des octets du texte chiffré, bloc par bloc, et en observant la réponse de l'oracle à chaque modification, un attaquant peut, sans jamais connaître la clé, **déchiffrer progressivement l'intégralité du message**, octet par octet, uniquement grâce à cette information binaire de validité du padding.

Cette classe d'attaque a été formellement décrite par **Serge Vaudenay en 2002**, et a depuis compromis, en pratique, de nombreuses implémentations réelles de TLS/SSL — l'exemple le plus connu étant l'attaque **POODLE** (2014), qui a exploité une variante de cette faiblesse contre le protocole obsolète SSL 3.0, contribuant à accélérer son retrait définitif des navigateurs et serveurs modernes. La leçon générale à en tirer, pour la conception de systèmes cryptographiques, est qu'un mode opératoire non authentifié (CBC seul, sans vérification d'intégrité associée) expose structurellement ce type de canal d'information, ce qui motive fortement, là encore, l'adoption de schémas de chiffrement authentifié (AES-GCM) qui détectent toute altération du texte chiffré avant même de tenter le déchiffrement, sans jamais exposer d'information partielle sur la validité du padding.

4. Attaques par analyse de consommation électrique et électromagnétique

Deux techniques complémentaires exploitent les variations physiques de consommation électrique d'un circuit exécutant un calcul cryptographique, sous l'hypothèse — vérifiée empiriquement dans de nombreux cas — que cette consommation dépend, au moins partiellement, des données et des opérations manipulées à chaque instant.

- **SPA (Simple Power Analysis, analyse simple de la consommation)** : cette technique consiste à observer **directement** une seule trace de consommation électrique enregistrée pendant l'exécution d'une opération cryptographique, en cherchant à y reconnaître visuellement des motifs correspondant à des opérations distinctes. Un exemple classique concerne l'exponentiation modulaire (utilisée dans RSA) : les opérations de mise au carré et de multiplication modulaire peuvent présenter des signatures de consommation électrique visuellement distinctes sur la trace enregistrée, ce qui, dans une implémentation non protégée, peut permettre à un observateur entraîné de déduire directement la séquence de bits de l'exposant secret utilisé.
- **DPA (Differential Power Analysis, analyse différentielle de la consommation)** : cette technique, plus puissante et plus robuste au bruit de mesure que la SPA, repose sur l'analyse **statistique** d'un grand nombre de traces de consommation enregistrées pour de nombreuses entrées différentes. En corrélant statistiquement les variations de consommation observées à des hypothèses sur des bits de clé candidats, un attaquant peut retrouver progressivement des portions de la clé secrète, même en présence d'un bruit de mesure important qui rendrait une analyse simple (SPA) inefficace.

Ces deux techniques concernent principalement des **dispositifs physiques** dont l'attaquant peut obtenir un accès physique direct ou de proximité suffisamment prolongé pour effectuer les mesures nécessaires — typiquement des cartes à puce, des modules matériels de sécurité (HSM), ou des objets connectés dont le boîtier reste accessible. Elles sont beaucoup moins pertinentes contre un service distant accessible uniquement par le réseau, où l'attaquant n'a par définition aucun accès physique au matériel d'exécution.

5. Attaques par cache (cache-timing attacks)

Cette famille d'attaques exploite les **différences de temps d'accès mémoire** entre une donnée présente dans le cache du processeur (accès rapide) et une donnée absente du cache, devant être récupérée depuis la mémoire principale (accès nettement plus lent). Ces différences, de l'ordre de quelques dizaines à quelques centaines de cycles processeur, sont mesurables avec une précision suffisante par un attaquant disposant d'un accès (même limité, par exemple sous forme d'un processus non privilégié partageant le même processeur physique) à la machine exécutant l'opération cryptographique ciblée.

Un cas concret et bien documenté concerne certaines implémentations logicielles d'**AES** ayant recours à des **tables précalculées** (dites *T-tables*), destinées à accélérer le calcul en substituant certaines opérations par de simples consultations de table indexées, en partie, par des bits de la clé secrète. Si l'accès à ces tables passe par le cache processeur de façon dépendante de la clé, un attaquant capable de mesurer les temps d'accès mémoire peut, par une analyse statistique similaire dans son principe à la DPA, retrouver progressivement des informations sur la clé utilisée. Cette classe de vulnérabilité a directement motivé, au niveau matériel, l'adoption d'**instructions dédiées** telles que le jeu d'instructions **AES-NI**, intégré nativement dans les processeurs modernes, qui exécute les opérations de chiffrement AES en temps constant, indépendamment de la valeur de la clé, éliminant structurellement ce canal d'attaque au niveau matériel plutôt que de compter sur une implémentation logicielle soigneusement écrite en temps constant.

6. Attaques sur les générateurs de nombres aléatoires

Un **générateur de nombres pseudo-aléatoires** (PRNG) de mauvaise qualité, ou correctement conçu mais **mal initialisé** — typiquement à cause d'une entropie insuffisante disponible au moment de son démarrage, ou d'une graine (*seed*) initiale prévisible — compromet **directement et intégralement** toute construction cryptographique qui en dépend en aval : génération de clés secrètes, génération de nonces, génération de vecteurs d'initialisation. Cette faiblesse est particulièrement insidieuse car elle peut coexister avec un algorithme cryptographique par ailleurs parfaitement robuste et correctement implémenté : le maillon faible se situe uniquement dans la source d'aléa, en amont de tout le reste du système.

Un cas réel largement documenté publiquement concerne une faille découverte dans le générateur de nombres aléatoires d'une distribution du système d'exploitation Debian (2006-2008), résultant d'une modification malencontreuse du code source qui avait, sans le vouloir, considérablement réduit l'entropie effective utilisée pour initialiser le générateur sur les systèmes affectés. Cette faille a eu pour conséquence directe de réduire l'espace des clés cryptographiques SSH générées sur les systèmes concernés à seulement quelques dizaines de milliers de valeurs distinctes possibles, un espace suffisamment restreint pour être intégralement énuméré, ce qui a nécessité une opération massive de régénération des clés affectées une fois la faille identifiée et corrigée. Ce cas illustre de façon exemplaire le principe évoqué au chapitre 1 : une erreur d'implémentation, même localisée dans un composant en apparence périphérique comme un générateur aléatoire, peut annuler entièrement la sécurité théorique d'un système cryptographique par ailleurs sain.

7. Démarche défensive face aux canaux auxiliaires

Face à cette diversité de menaces, une démarche défensive cohérente repose sur quelques principes directeurs simples mais essentiels à respecter systématiquement :

- **Utiliser des implémentations cryptographiques éprouvées et auditées** (bibliothèques largement reconnues et maintenues par la communauté ou des acteurs établis) plutôt que de réimplémenter soi-même des primitives cryptographiques sensibles : la résistance aux canaux auxiliaires demande une expertise très spécialisée, rarement acquise en dehors d'équipes dédiées à la cryptographie appliquée, et de nombreuses subtilités d'implémentation (temps constant, protection contre les fuites de cache) sont facilement négligées par une implémentation « maison » bien intentionnée mais non spécialisée.
- **Privilégier systématiquement les implémentations en temps constant** pour toute opération manipulant directement un secret (comparaison, opération arithmétique dépendant de bits de clé), en s'appuyant si possible sur des instructions matérielles dédiées (comme AES-NI) plutôt que sur des optimisations logicielles génériques dont le comportement temporel peut dépendre involontairement des données traitées.
- **S'assurer d'une source d'entropie fiable** pour toute génération aléatoire à usage cryptographique, en s'appuyant sur les mécanismes fournis par le système d'exploitation (`/dev/urandom` sur les systèmes de type Unix, ou l'équivalent système approprié) ou sur un générateur pseudo-aléatoire cryptographiquement sûr (CSPRNG) dédié, plutôt que sur un générateur pseudo-aléatoire générique non conçu pour un usage sécuritaire.
- **Pour les dispositifs physiques particulièrement sensibles** (cartes à puce, modules de sécurité matériels), envisager des contre-mesures matérielles dédiées, telles que des techniques de masquage (introduction de valeurs aléatoires intermédiaires dans le calcul pour décorrélérer la consommation observée des données réelles) ou l'ajout délibéré de bruit dans le circuit, afin de rendre les mesures physiques exploitées par SPA/DPA statistiquement plus difficiles à interpréter.

À retenir

- Les attaques par canaux auxiliaires exploitent l'implémentation d'un système cryptographique, jamais son algorithme mathématique en tant que tel : elles rappellent qu'une preuve mathématique de sécurité ne suffit pas, à elle seule, à garantir une sécurité pratique.
- Le padding oracle est une attaque pratique majeure contre les usages naïfs et non authentifiés de CBC (Vaudenay, 2002), à l'origine de plusieurs failles réelles documentées dans TLS/SSL, dont l'attaque POODLE (2014).
- Les attaques par consommation électrique (SPA/DPA) et par cache-timing ciblent principalement des implémentations non protégées en temps constant, ce qui a motivé l'adoption d'instructions matérielles dédiées comme AES-NI.
- Un générateur aléatoire de mauvaise qualité ou mal initialisé peut annuler entièrement la sécurité de tout système cryptographique qui en dépend, quelle que soit par ailleurs la robustesse théorique de l'algorithme utilisé — un cas documenté publiquement (Debian, 2006-2008) l'illustre concrètement.